

TECH TALK / APR 2026

AWS DevOps in Action

Building a production-grade CI/CD pipeline with CodePipeline, CodeBuild, and CodeDeploy

Presented by **Arya Singh**

Case study: intraday_trading — a serverless trading analytics platform

Why DevOps? The Three Pillars

Automate the path from code → production, safely and repeatably.

THE PROBLEM: Without automation, every deploy means manual terraform apply, manual S3 sync, manual CloudFront invalidation — slow, error-prone, no safety net, and no audit trail.

WHAT FIRES WHEN

CodePipeline

The Orchestrator

Connects GitHub → Build → Deploy. Defines stages, gates, and artifact flow. The conductor of the symphony.

RUNS YOUR CODE

CodeBuild

The Worker

Runs commands in ephemeral Linux containers — compiles, tests, plans Terraform, publishes artifacts. The muscle.

DELIVERS SAFELY

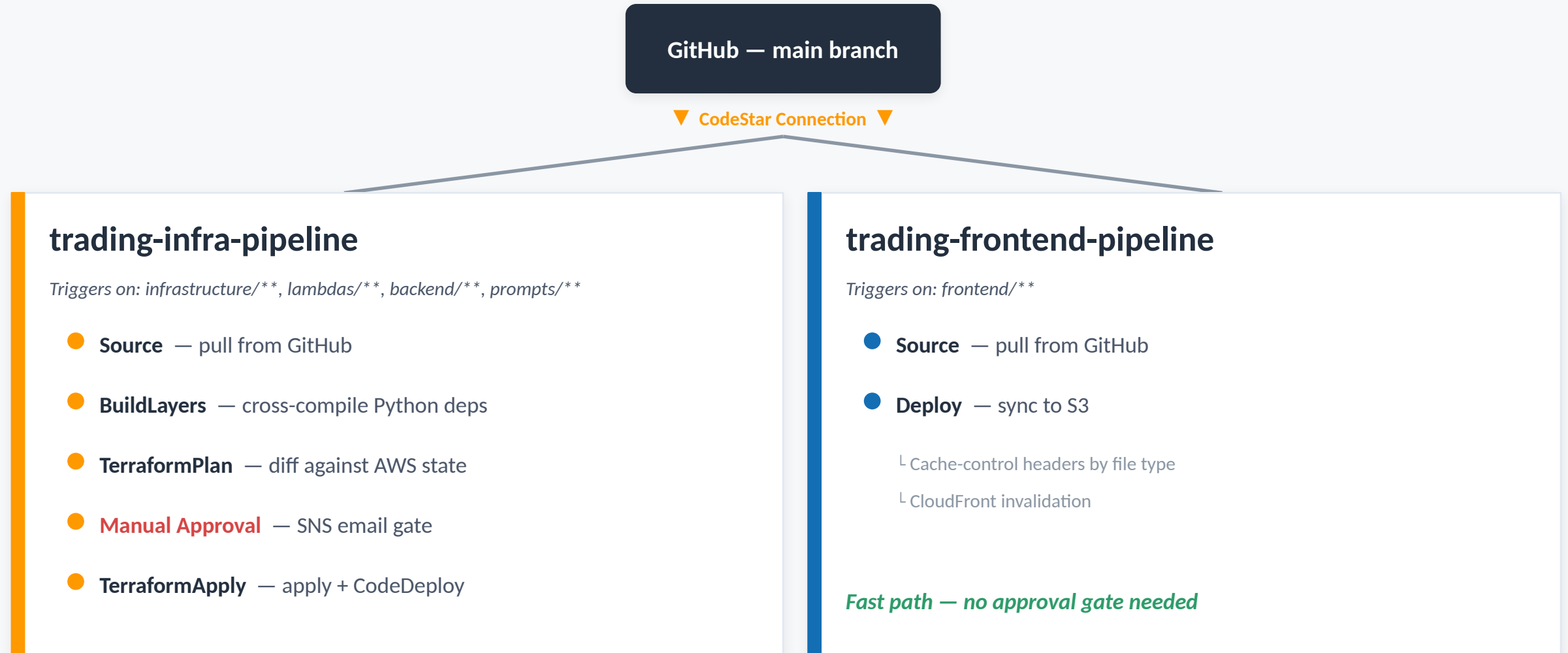
CodeDeploy

The Safe Lander

Shifts Lambda traffic gradually, watches CloudWatch alarms, auto-rollbacks on errors. The parachute.

Two Pipelines, One Repo

Path-based routing: infra and frontend changes have different risk profiles — they deserve different pipelines.



Inside CodeBuild — The Four Stages

Each stage runs in an ephemeral container. Artifacts flow through S3 — stages are decoupled and independently retryable.

01

BuildLayers

MEDIUM • 4 vCPU • ~8 min

Cross-compiles Python packages for Lambda's Linux x86_64 runtime. Cannot be done on a Mac — binaries must match.

Pip cache in S3 → ~6 min saved on re-runs

02

TerraformPlan

SMALL • 2 vCPU • ~3 min

Runs terraform plan against S3-backed state. Produces a binary tfplan diff — exactly what will change in AWS.

No changes applied yet — safe to review

03

Approval Gate

MANUAL • SNS email

Pipeline pauses. Reviewer checks the plan output in CloudWatch logs, then approves or rejects.

The human safety net for production infra

04

TerraformApply

SMALL • 2 vCPU • ~5 min

Applies the exact tfplan from stage 2 — no drift. Publishes Lambda versions and triggers CodeDeploy.

Exits by handing off to CodeDeploy

KEY INSIGHT: S3 artifacts decouple stages. If apply fails, the plan artifact is still there — retry without rebuilding.

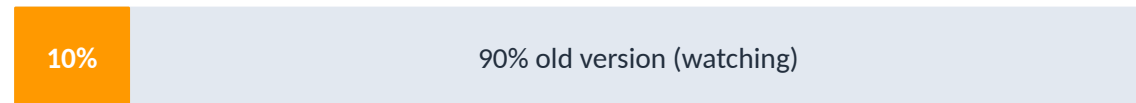
CodeDeploy — The Safety Net

Gradual traffic shifting + CloudWatch alarms = automatic rollback when bad code reaches production.

TRAFFIC SHIFT OVER TIME

CANARY — stocks-signal

10% → new version for 5 min → 100% shift



Fast feedback • Low blast radius • 5-min bake

LINEAR — options-analysis

+10% every 1 min → full shift over 10 minutes



Gradual rollout • More observation time • Less aggressive

AUTOMATIC ROLLBACK FLOW

- 1 Deployment begins**
New version gets traffic slice
- 2 CloudWatch watches**
Lambda Errors metric, 1-min windows
- 3 Threshold exceeded**
e.g. > 3 errors in 2 periods
- 4 Alarm fires**
CloudWatch → CodeDeploy
- 5 Auto-rollback**
100% traffic returns to old version

KEY TAKEAWAYS

Why This Architecture Wins



Fast & Reliable Triggers

V2 pipelines use EventBridge with path filters — infra changes don't retrigger frontend, and vice versa.



Safety First

Manual approval gate on infra. Canary deployments on critical Lambdas. Auto-rollback on CloudWatch alarms.



Fully Repeatable

Everything in Terraform — the pipeline defines itself. Destroy & recreate the entire stack from git.



Least Privilege

Three separate IAM roles — CodePipeline, CodeBuild, CodeDeploy. Compromise of one doesn't cascade.



Cost-Effective

V2 pay-per-minute pricing: ~\$0.70/month for our usage vs \$2/month flat on V1. Pipelines pay for what they do.



Observable

CloudWatch logs per build. SNS notifications on approval. X-Ray traces on Lambdas. Everything is visible.

From git push to production, safely — in under 20 minutes.